

 RAG ENGINEERING

Silent Failure Detection in RAG

Your RAG pipeline returns HTTP 200 every time. Sometimes the answer is confidently wrong. Your monitoring never flagged it.

6 failure types that hide behind a green status page

 BAD RETRIEVAL

 HALLUCINATION

 STALENESS

 OUT-OF-SCOPE

6 defensive layers. Real Python code. Production-ready thresholds.



AI Coach John

Follow



PROITBRIDGE
Strive For Better Future

 Repost

What Is a Silent Failure?

A silent failure returns a confident, well-formatted answer that is factually wrong, hallucinated, stale, or irrelevant. No exception. No 5xx. No alert fires. Users silently receive bad information.

💀 Why RAG Is Especially Vulnerable

RAG layers LLM creativity on top of retrieval. If retrieval is off by a small cosine margin, the LLM confidently synthesizes an answer from wrong context. The response sounds authoritative and complete.

👤× Real-World Consequences

Legal teams relying on stale compliance documents. Customers getting wrong pricing or product specs. Automated pipelines consuming hallucinated data as ground truth. Businesses facing regulatory risk because a RAG answered incorrectly.

🏗️ The Only Solution

Silent failures require proactive multi-layer detection built into the pipeline itself. You cannot rely on user complaints because most users trust the system and never report issues.



AI Coach John

Follow



PROITBRIDGE
Strive For Better Future

↻ Repost



The 6 RAG Silent Failure Types

Failure	Root Cause	Primary Signal	Severity
Bad Retrieval	Low cosine similarity to query	Retrieval score below 0.5	Critical
Hallucination	LLM adds facts not in context	NLI contradiction flag	Critical
Stale Content	Source docs outdated	updated_at metadata age	High
Out-of-Scope	Query outside knowledge base	Max corpus similarity	High
Context Conflict	Retrieved docs disagree	Pairwise NLI contradiction	Medium
Low Confidence	Model uncertain, answers anyway	Token log-probability	Medium



Each failure type needs a dedicated detection layer. No single check catches all six. You must stack them.



AI Coach John

Follow



PROITBRIDGE
Strive For Better Future

Repost

Your 6-Layer Detection Stack

1

Retrieval Relevance

bge-small cosine score. Decline if top score below 0.5

2

Hallucination via NLI

deberta-v3-small: detect answer-context contradiction

3

Temporal Staleness

Regex + updated_at metadata: flag time-sensitive queries

4

Out-of-Scope Detection

Max corpus similarity. Decline gracefully if below threshold

5

Context Conflict Check

Pairwise NLI on retrieved chunks. Flag contradicting sources

6

Per-Query Tracing

Structured trace for every request. Full audit trail



AI Coach John

Follow



PROITBRIDGE
Strive For Better Future

 Repost

Retrieval Relevance Scoring

Check whether retrieved chunks are actually relevant to the query before calling the LLM. Low relevance means near-certain hallucination downstream.

```
from sentence_transformers import SentenceTransformer, util

encoder = SentenceTransformer("BAAI/bge-small-en-v1.5")

def retrieval_relevance_score(query: str, chunks: list[str]) → dict:
    q_emb = encoder.encode([query], normalize_embeddings=True)
    c_emb = encoder.encode(chunks, normalize_embeddings=True)
    scores = util.dot_score(q_emb, c_emb)[0].tolist()
    avg, top = sum(scores)/len(scores), max(scores)
    return {"avg": avg, "top": top,
            "low_relevance": top < 0.50,
            "per_chunk": scores}

result = retrieval_relevance_score(query, retrieved_chunks)
if result["low_relevance"]:
    return "I could not find relevant information for your question."
```

⚙️ Thresholds

Top score above 0.7: high confidence.
0.5 to 0.7: marginal. Below 0.5:
decline. Calibrate on your dataset.

⌚ Latency

bge-small adds only 8 to 15ms on CPU.
Run before the LLM call. Never skip
this check.



Hallucination via NLI Contradiction

Use a Natural Language Inference model to check whether the generated answer is entailed by the context or contradicts it. Contradiction = hallucination signal.

```
from transformers import pipeline

nli = pipeline("text-classification",
               model="cross-encoder/nli-deberta-v3-small")

def detect_hallucination(answer: str, context_chunks: list[str]) → tuple:
    contradictions = []
    for chunk in context_chunks:
        result = nli(f"{chunk[:400]} [SEP] {answer[:300]}")[0]
        if (result["label"] == "CONTRADICTION"
            and result["score"] > 0.85):
            contradictions.append({
                "chunk_preview": chunk[:80],
                "confidence": result["score"]
            })
    hallucinated = len(contradictions) > 0
    return hallucinated, contradictions
```

🔧 Model Choice and Speed

cross-encoder/nli-deberta-v3-small: 40ms on GPU, accurate for short texts.

facebook/bart-large-mnli: slower but better for long passages. Run async in batch for offline evaluation with RAGAS faithfulness.



Temporal Staleness Detection

Queries containing time-sensitive signals (prices, dates, regulations) must be checked against document freshness. Old documents answer current questions incorrectly with confidence.

```
import re
from datetime import datetime, timezone

TIME_SENSITIVE_PATTERNS = [
    r\b(price|cost|fee|rate|salary)\b,
    r\b(latest|current|recent|now|today|2024|2025)\b,
    r\b(regulation|compliance|policy|law|act)\b,
    r\b(available|in\s+stock|shipping|release)\b,
]

def check_temporal_staleness(query: str, doc_metadata: dict,
                             stale_days: int = 90) → bool:
    time_sensitive = any(
        re.search(p, query, re.IGNORECASE)
        for p in TIME_SENSITIVE_PATTERNS
    )
    if not time_sensitive:
        return False
    updated = doc_metadata.get("updated_at")
    if not updated:
        return True # no timestamp = assume stale
    age_days = (datetime.now(timezone.utc) - updated).days
```



Store updated_at on every document at ingest time. This costs zero extra storage and enables this entire detection layer.



AI Coach John

Follow



🔄 Repost

Out-of-Scope Query Detection

When a query falls outside your knowledge base, the LLM still generates an answer. That answer is nearly always hallucinated. Detect and decline gracefully before calling the LLM.

```
import numpy as np

class OutOfScopeDetector:
    def __init__(self, corpus_embeddings: np.ndarray, threshold: float = 0.42):
        self.corpus = corpus_embeddings  # shape (N, embedding_dim)
        self.threshold = threshold

    def is_out_of_scope(self, query_embedding: np.ndarray) → tuple[bool, float]:
        # max cosine similarity to any document in the knowledge base
        similarities = np.dot(self.corpus, query_embedding)
        max_sim = float(similarities.max())
        oos = max_sim < self.threshold
        return oos, max_sim

detector = OutOfScopeDetector(corpus_embs, threshold=0.42)
oos, score = detector.is_out_of_scope(q_embedding)
if oos:
    return "This topic is outside my current knowledge scope."
```

Implementation Note

Pre-compute corpus embeddings at startup. Store as numpy array in memory. The dot product scan adds under 5ms for 100k documents at embedding dim 384. No recomputation needed until re-index.



Context Conflict Detection (Pairwise NLI)

Retrieved chunks sometimes contradict each other. The LLM then synthesizes a confused or blended answer. Detect inter-chunk contradictions before generation.

```
from itertools import combinations

def detect_context_contradictions(chunks: list[str],
                                  nli_model,
                                  threshold=0.75) → list[dict]:
    conflicts = []
    for (i, c1), (j, c2) in combinations(enumerate(chunks), 2):
        r = nli_model(f"{c1[:300]} [SEP] {c2[:300]}")[0]
        if r["label"] == "CONTRADICTION" and r["score"] > threshold:
            conflicts.append({
                "chunk_a": i, "chunk_b": j,
                "score": r["score"]
            })
    return conflicts

conflicts = detect_context_contradictions(retrieved_chunks, nli)
if conflicts:
    # Remove lower-scored chunk from pair, or add caveat to answer
    handle_conflicts(conflicts, retrieved_chunks)
```

! $O(n^2)$ pairs but N is typically 3 to 8 chunks. At N=8 that is 28 NLI calls, under 200ms total on GPU.



AI Coach John

Follow



 Repost

Async RAGAS Quality Evaluation

RAGAS provides four automated metrics for RAG quality. Run offline in batch on a sample of production queries to get a comprehensive quality snapshot.

```
from ragas import evaluate
from ragas.metrics import (faithfulness, answer_relevancy,
                           context_recall, context_precision)
from datasets import Dataset

eval_data = {
    "question": [q for q, _, _ in prod_sample],
    "contexts": [ctx for _, ctx, _ in prod_sample],
    "answer": [a for _, _, a in prod_sample],
    "ground_truth": [g for _, _, _, g in prod_sample],
}

dataset = Dataset.from_dict(eval_data)
result = evaluate(dataset,
                  metrics=[faithfulness, answer_relevancy,
                           context_recall, context_precision])

# faithfulness below 0.80 → hallucination risk
# context_recall below 0.70 → retrieval missing key info
```



Run RAGAS weekly on a 100-query sample from production. Track all 4 metrics as a time series. Alert when any drops 10% week over week.



AI Coach John

Follow



PROITBRIDGE
Strive For Better Future

 Repost

Full Traced RAG Pipeline

```

async def traced_rag_pipeline(query: str, user_id: str) → dict:
    trace = {"query": query, "user_id": user_id,
            "ts": datetime.utcnow().isoformat()}

    chunks, meta = await retrieve_with_metadata(query)
    trace["retrieval_score"] = retrieval_relevance_score(query, chunks)
    trace["staleness"]      = check_temporal_staleness(query, meta)
    oos, oos_score          = detector.is_out_of_scope(q_emb)
    trace["out_of_scope"]   = oos
    conflicts                = detect_context_contradictions(chunks, nli)
    trace["conflicts"]      = conflicts

    if oos or trace["retrieval_score"]["low_relevance"] or trace["staleness"]:
        trace["declined"] = True
        await log_trace(trace)
        return {"answer": FALLBACK_MSG, "trace": trace}

    answer = await generate(query, chunks)
    halluc, _ = detect_hallucination(answer, chunks)
    trace["hallucination"] = halluc
    await log_trace(trace)
    return {"answer": answer, "trace": trace}

```



Every trace includes all 6 signals. When a user reports a bad answer, you can replay the trace and see exactly which layer failed and why.



AI Coach John

Follow



PROITBRIDGE
Strive For Better Future

 Repost

ALERT THRESHOLDS

Production Alerting Thresholds

Signal	Warn	Critical	Response
Avg retrieval score	Below 0.60	Below 0.45	Re-index FAISS
Hallucination rate	Above 5%	Above 10%	Prompt engineering review
Staleness rate	Above 10%	Above 25%	Document refresh pipeline
Out-of-scope rate	Above 15%	Above 30%	Expand knowledge base
Conflict rate	Above 8%	Above 20%	Document deduplication
Decline rate total	Above 10%	Above 20%	Full pipeline audit
RAGAS faithfulness	Below 0.80	Below 0.65	Review generation prompt



Daily: OOS rate, decline rate, retrieval score. Weekly: RAGAS batch eval on 100 sampled queries. Monthly: full pipeline health review.



AI Coach John

Follow



PROITBRIDGE
Strive For Better Future

 Repost

6 RAG Monitoring Mistakes

- ✗ **Monitoring only the LLM call.** Retrieval failure is invisible at the LLM layer. Bad retrieval causes hallucination. Monitor retrieval first, every single call.
- ✗ **No updated_at timestamp on documents.** Without it, you cannot perform any temporal freshness check. Store it at ingest time. No exceptions.
- ✗ **Relying only on user feedback.** Most users never report wrong answers. They just stop using the product. Automated signals are mandatory.
- ✗ **Treating high decline rate as a bug.** A high OOS rate means your knowledge base needs expansion, not that the detection system is broken.
- ✗ **No golden query regression set.** Without 50 known-good queries, you cannot detect when a pipeline change silently breaks existing correct behavior.
- ✗ **Checking NLI on full documents instead of chunks.** NLI models have token limits. Truncate input to 400 tokens per chunk for accurate scores.



RAG Silent Failure **Full Cheatsheet**

Layer	Tool / Method	Key Threshold
Retrieval quality	bge-small cosine score	Top score above 0.50
Hallucination	deberta-v3-small NLI	No contradiction above 0.85
Staleness	updated_at + regex patterns	Age below 90 days
Out-of-scope	Max corpus similarity scan	Max similarity above 0.42
Context conflict	Pairwise NLI on chunks	No contradiction above 0.75
Full quality eval	RAGAS (weekly batch)	Faithfulness above 0.80
Per-query tracing	Structured trace log	100% coverage, no exceptions

 **Check Retrieval first. Decline early. Trace everything. No trace = no root cause analysis.**



AI Coach John

Follow



 Repost

 KEY TAKEAWAY

Your RAG Pipeline Needs Proactive Defenses

HTTP 200 does not mean correct. Silent failures are the hardest bugs in AI systems because they are invisible until someone is already harmed.

✓ Your 3 Non-Negotiable Starting Points

- Add retrieval relevance scoring to every RAG call today
- Store `updated_at` on every document at ingest time
- Log a structured trace for every single query to enable root cause analysis

Follow AI Coach John for more production AI engineering content.



AI Coach John

Follow



PROITBRIDGE
Strive For Better Future

 Repost